Demystifying the Target – APIs, REST, and Data Formats

The Communication Shift

Historically, network administration has relied on the Command Line Interface (CLI). The CLI is designed for human-to-machine communication; it provides prompts, error messages, and visual formatting. However, automation engines do not "type" into a CLI. They require a structured, highly predictable method of machine-to-machine communication. This is achieved using Application Programming Interfaces (APIs).

APIs and the REST Architecture

An API is a defined interface that allows one software system to interact with another. It acts as a gateway, defining the types of requests that can be made, how they should be formatted, and the conventions for responses.

In modern networking, the industry standard is the REST API (Representational State Transfer). REST is an architectural style that operates over standard web protocols (HTTP/HTTPS). When an administrator interacts with the Cisco vManage GUI, the web browser is actually sending hidden REST API calls to the server. In an Infrastructure-as-Code environment, the automation tools bypass the GUI and send those exact same API calls directly to the controller.

REST Methods (The Verbs)

REST APIs utilize standard HTTP methods to transfer data in and out of a system. These methods dictate the action the controller should take. The four primary methods used in network automation are:

- **GET (Read):** Retrieves data from the controller. This is the API equivalent of running a show command. It does not alter the configuration; it simply requests the current state or operational data.
- **POST (Create):** Sends new data to the controller to create a new resource. This is used when deploying a new policy, template, or device that does not currently exist in the system.
- **PUT / PATCH (Update):** Modifies an existing resource. If a BGP template already exists and an administrator needs to update a timer or add a neighbor, a PUT or PATCH request is sent containing the updated data.

Cisco Confidential

- **DELETE (Remove):** Instructs the controller to remove a specific resource, such as deleting an obsolete routing policy.

JSON: The Language of Machines

When systems communicate via REST APIs (such as sending a POST request to create a policy), they must agree on a data format for the payload. The universal standard for this is JSON (JavaScript Object Notation).

JSON is a lightweight data-interchange format. It is highly structured, unambiguous, and easily parsed by machines. Data in JSON is organized into two primary structures:

1. **Key-Value Pairs:** Data is represented as a defined key linked to a specific value (e.g., "hostname": "RTR-01").
2. **Arrays (Lists):** Ordered lists of values, enclosed in square brackets, used when a key has multiple attributes (e.g., a list of multiple BGP neighbors).

While JSON is perfect for programmatic parsing, it is notoriously unforgiving for humans to author. It requires a strict syntax of curly braces, square brackets, and quotation marks. A single missing comma renders the entire payload invalid. Furthermore, JSON does not natively support comments, making it difficult to document the rationale behind a configuration within the file itself.

YAML: The Language of Humans

To bridge the gap between human readability and machine structure, the Infrastructure-as-Code industry adopted YAML (YAML Ain't Markup Language).

YAML represents the exact same data structures as JSON (key-value pairs, arrays, and objects) but strips away the rigid punctuation. Instead of brackets and commas, YAML relies strictly on whitespace and line indentation to define the hierarchy of the data.

Additionally, YAML supports inline comments (using the # symbol). This is a critical feature for NetDevOps, as it allows administrators to annotate configurations, explain complex policies, and leave notes for peer reviewers directly inside the code. Functionally, YAML and JSON are completely interchangeable; any valid YAML file can be programmatically translated into JSON.

Cisco Confidential

The SDWAN-as-Code Workflow

Understanding the relationship between these elements is critical for grasping the SDWAN-as-Code pipeline. Administrators do not manually write raw API calls, nor do they author JSON payloads. The workflow operates as follows:

1. **The Human:** The administrator defines the desired state of the network by writing human-readable, commented **YAML** files.
2. **The Translation:** An automation engine (like Terraform) reads the YAML files and automatically translates them into machine-readable **JSON**.
3. **The Execution:** The automation engine transmits that JSON payload to the Cisco vManage controller via a **REST API** using the appropriate method (POST, PUT, or DELETE).
4. **The Result:** vManage translates the API payload into the proprietary device configuration and pushes it to the physical edge routers.

Cisco Confidential